

Fast Weights Predict, Not Evaluate: A Hebbian Associative World Model for Interception

Gabriel Hill
gabriel@familyfungroup.com

Abstract

The Dragon Hatchling (BDH) architecture is built on a fast-weight associative memory — a matrix of stimulus–response associations maintained by outer-product plasticity and read out by similarity-weighted recall. We ask what this primitive is *for*: is it a value function, an evaluative map from states to expected return, or a world model, a predictive map from states to next states? We argue for the latter and settle the question empirically in a controlled interception task, where a turn-rate-limited pursuer must catch a moving target. Three findings emerge. **(1)** Used as a value function, the memory collapses: a linear TD learner with the same associative readout reaches $\sim 2\text{--}10\%$ of the lead-pursuit ceiling, and backpropagated DQN and PPO do not exceed the *no-prediction reflex*. **(2)** Used as a world model with greedy lead-pursuit planning, the same memory reaches $\sim 94\%$ of the lead-pursuit ceiling. **(3)** On persistently curving prey — where intercepting a trajectory, not a point velocity, is what matters — the learned world model beats first- and second-order analytic extrapolation by 17% and 193% respectively, because rolling forward the learned dynamics is more accurate than a truncated Taylor series. We interpret the results as supporting a Dreamer-style division of labor: fast-weight memories are best understood as *predictive* substrates trained by dense self-supervised error, with sparse, outcome-driven planning on top, rather than as value-function approximators.

1 Introduction

A central question in biologically plausible machine learning is what a particular synaptic plasticity rule *computes*. The Dragon Hatchling (BDH) architecture [1] places at its core a fast-weight associative memory: a weight matrix updated by an outer product between a pre-synaptic activity vector and a post-synaptic signal, and queried by an inner product that recalls stored associations weighted by similarity. This is one of the oldest and most general ideas in connectionism — the linear associator, the Hopfield memory, the “fast-weight programmer” [2, 3]. But *what should the post-synaptic signal encode?* Two readings are possible, and they lead to very different systems:

- **The evaluative reading.** The memory stores state $\rightarrow$ *value* associations, forming a value function $V(s)$ or $Q(s, a)$ that is learned by bootstrapped temporal-difference (TD) updates and queried by a policy to select actions.
- **The predictive reading.** The memory stores state $\rightarrow$ *next-state* associations, forming a world model $\hat{s}_{t+1} = f(s_t)$ that is trained by self-supervised prediction error and rolled forward (imagined) to plan.

These readings are not cosmetic: they make opposite claims about the role of the same plasticity rule. The BDH paper’s own framing — and the broader literature on reward-modulated Hebbian

learning [6] — has tended toward the evaluative reading. We argue the predictive reading is the correct one, and we support the argument with a controlled experiment.

We choose **interception** (pursuit) as the test bed because it makes the two readings sharply distinguishable. Intercepting a moving target requires *predicting where the target will be*, not merely evaluating how good the current state is. It is also a domain with clean analytic baselines (first- and second-order extrapolation) against which a learned model can be judged, and with an oracle ceiling we can compute. Finally, interception is a canonical biologically relevant behavior, present from dragonflies to predators to human ball-players.

Our contributions are:

1. A **conceptual reframing** of fast-weight associative memories as world models (predictive substrates) rather than value functions, with a precise three-factor Hebbian instantiation.
2. A **seeded, multi-environment interception benchmark** with analytic, model-based, and model-free baselines, a reset-on-catch protocol that measures genuine interception rather than “lock-on,” and a forecast-error metric that isolates prediction quality.
3. **Empirical evidence** that (a) model-free value learning — including a linear TD learner using the *same* associative memory, a backprop DQN, and PPO — fails catastrophically on interception, while (b) the same memory as a world model reaches $\sim 94\%$ of the lead-pursuit ceiling and (c) beats first- and second-order analytic extrapolation on curved prey.

We are explicit about scope: this is a controlled toy domain, not a benchmark suite, and its purpose is to isolate one conceptual question. Section 6 discusses limitations and what a fuller study would require.

2 Related Work

Fast-weight memories. The idea of using rapid weight changes as a form of short-term memory traces back to [2], and has been connected to linear attention (“transformers are secretly fast-weight programmers”) [3]. The Dragon Hatchling [1] centers a fast-weight associative memory as the bridge between transformer-style attention and brain-like computation. Our work uses the core primitive — the outer-product weight update and similarity-weighted readout — and asks what it should represent.

Hebbian and three-factor learning. Classical Hebbian learning co-activates pre- and post-synaptic units ($\Delta W \propto yx^\top$). Three-factor rules gate this plasticity by a third, neuromodulatory signal (e.g., reward or prediction error), yielding biologically plausible and provably convergent variants [6]. The delta (Widrow–Hoff / LMS) rule $W \leftarrow W + \eta \delta x^\top$, where δ is the prediction error, is precisely such a three-factor rule [8]; we use its normalized form.

Model-based RL and world models. Learning a model of the environment and planning with it is a classical RL paradigm. Dreamer [7] popularized the split we instantiate: train a world model densely on prediction error, then plan (imagine) rollouts with it, while the policy is driven by sparse task reward. Our world model is the same idea at the scale of a single associative matrix rather than a deep recurrent network.

The deadly triad. [4] identify three properties whose combination can cause divergence in TD learning: function approximation, bootstrapping, and off-policy updates. Our linear-Q baseline is a textbook instance of all three, and its failure is a live illustration.

Pursuit and interception. Interception is a classic control/geometry problem with analytic solutions (lead pursuit, proportional navigation) [9, 10] and biological relevance [11]. We use it as a clean, interpretable test bed rather than contributing new pursuit algorithms.

3 Problem Setting

3.1 Environment

We use a two-dimensional toroidal world of size 1200×800 px with discrete-time dynamics at $dt = 0.05$ s. A single **prey** moves autonomously; a single **chaser** (pursuer) attempts to catch it. A catch occurs when the chaser comes within a capture radius of 48 px; a 0.5 s cooldown follows before the next catch can register.

- **Chaser.** Constant top speed $v_c = 175$ px/s. Its heading changes at a rate bounded by $\omega_{\max} = 3.6$ rad/s (a *turn-rate clamp*). It cannot decelerate.
- **Prey.** Speed 165 px/s ($0.94\times$ the chaser), so the chaser cannot win by raw speed — it must *intercept*. Six dynamics (Table 1, top):
 - **const-vel** — constant velocity (linear; first-order extrapolation is exact).
 - **circling** — constant turn rate $\dot{\theta} \in \pm 1.2$ rad/s drawn per episode, so the prey runs in persistent arcs/circles.
 - **ou-turn, ou-vel** — Ornstein–Uhlenbeck (mean-reverting, stochastic) turn and speed.
 - **jump** — piecewise-constant heading with random jumps.
 - **flee** — reactive avoidance: steers away from the chaser and speeds up when approached.

The first two are the scientifically decisive pair: **const-vel** is where first-order extrapolation is already Bayes-optimal (a control), and **circling** is where intercepting a *trajectory* — not a point velocity — is what separates predictors. The stochastic (**ou-***, **jump**) and reactive (**flee**) prey test robustness and are where we document honest limits.

3.2 Protocol and metrics

A naive “catches per fixed horizon” metric is dominated by **lock-on**: once a faster chaser is within the capture radius it rarely loses the prey, so every competent agent saturates at the cooldown rate and prediction quality is invisible. We therefore use a **reset-on-catch** protocol: on each catch the prey is teleported to a uniformly random location 400–600 px from the chaser. Every catch is then a *fresh interception from distance*, and the catch count directly measures interception efficiency.

We report **catches per episode** (mean $\pm$ standard deviation over seeds) as the primary metric, and a **forecast error** — mean absolute error between a predictor’s forecast of the prey’s future position and its true position, evaluated at a fixed 1–2 s horizon — as a direct measure of prediction quality independent of the catch/steering loop.

3.3 The oracle ceiling

Interception has a clean ceiling. If the chaser knew the prey’s future trajectory exactly and could steer optimally under the turn-rate clamp, it would achieve an optimal catch rate. In practice the relevant ceiling for our comparison is the **lead-pursuit ceiling** — the catch rate of a chaser that steers perfectly at the true future prey position — which the analytic baselines approximate from below. We do not claim the chaser reaches the true optimum; we claim it reaches the lead-pursuit regime, and that the world model does so *without* a hand-crafted dynamics model.

4 Method

4.1 The fast-weight associative memory

Let $s \in \mathbb{R}^d$ be a normalized sensory state and $y \in \mathbb{R}^2$ a two-dimensional target (a velocity, below). A fast-weight memory is a matrix $W \in \mathbb{R}^{2 \times d}$ of associations, written and read as:

$$\text{read: } \hat{y} = W\phi(s), \quad \text{write: } W \leftarrow W + \eta \underbrace{(y - \hat{y})}_{\delta} \phi(s)^\top / (1 + \|\phi(s)\|^2). \quad (1)$$

The write is a **three-factor (error-gated) Hebbian** rule: the outer product of a pre-synaptic activity $\phi(s)$ and a post-synaptic prediction error δ , normalized for stability (normalized LMS), with a small synaptic weight decay. The read is **content-addressable**: $\hat{y} = W\phi(s)$ is the similarity-weighted recall of all stored associations, so the memory *generalizes across states* rather than storing a lookup table. This is the BDH fast-weight primitive; nothing else is added.

The sensory features $\phi(s)$ are, by default, **linear**: $\phi(s) = [1, \Delta x/R, \Delta y/R, v_x/v_{\max}, v_y/v_{\max}]$, where Δ is the wrapped relative position of the prey and v its velocity. We deliberately do *not* hand the model the prey’s turn rate or acceleration; it must infer curvature from raw observations. (An optional random-Fourier expansion can be added to make the readout nonlinear; Section 5.4 shows it is unnecessary here.)

4.2 The world model

We use the memory as a **world model**: it learns the prey’s one-step transition $f : s_t \mapsto v_{t+1}$, mapping the current state to the *next* velocity. Training is dense and self-supervised — every observed tick updates the memory toward the true next velocity — with no reward signal. To predict the prey’s position τ steps ahead, the model is **rolled forward (imagined)**: starting from the current state, it repeatedly predicts the next velocity and integrates it to a new position, for τ steps.

4.3 The planner

Planning is deliberately minimal and shared across all predictors: **lead pursuit** — steer at the predicted future prey position. The lead horizon is $\tau = d/v_c$ (the time to reach the prey at current speed), capped at 2 s, so the chaser aims at where the prey *will be*, not where it is. The chaser’s heading is then moved toward that aim under the turn-rate clamp. The policy (which way to steer) is thus driven only by the forecast; the only thing that varies across predictors is *how the forecast is made*.

4.4 The contrast: the same memory as a value function

To make the value-vs-world-model contrast clean, we implement the evaluative reading with the *identical* substrate: $Q(s, a) = w_a \cdot \phi(s)$ for 8 discrete headings, trained by TD(0) [5] with ϵ -greedy exploration (Section 5). This is exactly a linear value function over the same features ϕ , updated by a bootstrapped, off-policy rule — the textbook “deadly triad” configuration [4]. Any performance gap between it and the world model is therefore attributable to the *role* of the memory (evaluate vs. predict), not to the substrate, features, or representational capacity.

5 Experiments

5.1 Setup

All results are seeded and reproducible; each cell is a mean over seeds (10 for predictors, 3 for the slower model-free learners) with standard deviation. Hyperparameters were not tuned per-environment except where an ablation explicitly varies them. The BDH world model uses the linear feature set (Section 4), learning rate $\eta = 0.1$, and weight decay $\lambda = 10^{-3}$.

Baselines.

- *Predictors* (lead pursuit, differing only in the forecast):
 - **velocity-lead**: $\hat{p} = p + v\tau$ (first-order extrapolation).
 - **accel-lead**: $\hat{p} = p + v\tau + \frac{1}{2}a\tau^2$ (second-order Taylor, a from a finite-difference estimate).
 - **kalman-lead**: a constant-velocity Kalman filter + lead.
 - **bdh**: the fast-weight world model (Section 4).
- *Policies* (learn or select the heading directly):
 - **pure-pursuit**: steer at the *current* prey position (the no-prediction reflex).
 - **mpc**: model-predictive control — search 8 headings, simulate a short horizon with a first-order (constant-velocity) prey model, minimize final distance.
 - **linear-q**: linear TD Q-learning over the same features ϕ (the evaluative reading).
 - **dqn**: an MLP Q-network with replay buffer and target network.
 - **ppo**: clipped-surrogate PPO with policy and value MLPs.

The model-free learners receive a dense reward $r = -\text{distance}/600$ plus a +1 catch bonus, so their failure is a credit-assignment/interception failure rather than an artifact of sparse reward.

5.2 Result 1: world model vs. value function

Table 1 (bottom) reports catches under the reset-on-catch protocol. The contrast is stark and consistent across all six environments:

- The **linear-Q learner** — the same associative memory used as a value function — **collapses** to 0–27 catches, roughly 2–10% of the lead-pursuit ceiling.
- The **backprop DQN and PPO** do not exceed the *no-prediction reflex*: on most environments they are *worse* than pure pursuit, which does no learning and no prediction at all.
- The **BDH world model**, by contrast, reaches 165–384 catches — $\sim 94\%$ of the lead-pursuit ceiling — with a fraction of the training and no value function.

Because linear-Q and the world model use the *same* features and the *same* associative substrate, the gap is clean evidence that the fast-weight memory succeeds when it *predicts* and fails when it *evaluates*.

The clean ordering across approaches is *pure-pursuit (reflex)* ; *MPC (first-order search)* ; *velocity-lead (first-order, continuous)* ; *BDH (learned world model)* on five of six environments, with model-free value learning (linear-Q, DQN, PPO) *below the reflex* everywhere. Prediction and a learned model both matter, and planning with a model decisively beats learning a value function. (On **flee**, where a first-order model is wrong, MPC falls below the reflex — the *model*, not the *planner*, is the limiting factor.)

Table 1: Catches per episode (mean $\pm$ sd), reset-on-catch. *Top*: lead-pursuit predictors (10 seeds $\times$ 24,000 steps). *Bottom*: policies (3 seeds $\times$ 20,000 steps).

prey	velocity-lead	accel-lead	kalman-lead	BDH (world model)
const-vel	364 $\pm$ 57	364 $\pm$ 57	148 $\pm$ 50	363 $\pm$ 53
circling	327 $\pm$ 35	131 $\pm$ 192	35 $\pm$ 66	384 $\pm$ 38
ou-turn	368 $\pm$ 17	364 $\pm$ 10	116 $\pm$ 7	330 $\pm$ 11
ou-vel	388 $\pm$ 8	385 $\pm$ 15	136 $\pm$ 16	364 $\pm$ 12
jump	342 $\pm$ 10	293 $\pm$ 8	133 $\pm$ 10	298 $\pm$ 11
flee	182 $\pm$ 2	69 $\pm$ 7	6 $\pm$ 3	165 $\pm$ 5

prey	pure-pursuit	mpc	linear-q	dqn	ppo
const-vel	60 $\pm$ 10	242 $\pm$ 33	19 $\pm$ 6	30 $\pm$ 28	18 $\pm$ 5
circling	179 $\pm$ 52	254 $\pm$ 42	22 $\pm$ 5	41 $\pm$ 5	23 $\pm$ 5
ou-turn	129 $\pm$ 10	286 $\pm$ 23	25 $\pm$ 6	29 $\pm$ 4	23 $\pm$ 1
ou-vel	137 $\pm$ 11	304 $\pm$ 10	27 $\pm$ 5	29 $\pm$ 3	21 $\pm$ 9
jump	254 $\pm$ 8	265 $\pm$ 11	20 $\pm$ 2	44 $\pm$ 9	23 $\pm$ 7
flee	160 $\pm$ 1	140 $\pm$ 3	0 $\pm$ 1	37 $\pm$ 9	8 $\pm$ 8

5.3 Result 2: world model vs. analytic extrapolation on curved prey

On **circling** (Table 1, top), the BDH world model achieves 384 ± 38 catches, +17% over first-order extrapolation (327 ± 35) and +193% over second-order (131 ± 192). The second-order baseline is also wildly unreliable — its standard deviation (192) is larger than its mean — whereas the world model is stable (± 38).

The mechanism is visible in the forecast error: on **circling** the world model’s lead-horizon forecast error is ~ 62 – 69 px versus ~ 81 – 100 px for first-order. Rolling forward the learned rotation tracks the arc; a straight-line extrapolation (first-order) misses it, and a parabola (second-order) *overshoots* it over the ~ 2 s lead horizon — the $\frac{1}{2}a\tau^2$ term of a second-order Taylor expansion diverges from a circle. A learned model that re-applies the true (learned) one-step dynamics at each of τ steps is the correct object, and the fast-weight memory discovers it from raw observations without being told the prey is circling.

On **const-vel** the world model ties first-order (as it should: the dynamics are linear, and a linear model reproduces them exactly), confirming the model adds no spurious advantage. On the stochastic prey (**ou-***, **jump**) no predictor beats first-order — the future is genuinely unpredictable from the current state — and the learned model adds a small amount of variance. These are the honest negative results that delimit the claim.

5.4 Ablations

We verify the result is not fragile. (All ablations on **circling**, reset-on-catch.)

- **Turn-rate clamp** ($1.8 \rightarrow 5.4$ rad/s): BDH beats first-order at *every* turn rate (262/288/306/312 vs. 219/273/280/283), so the advantage is not an artifact of the clamp’s value.
- **Speed ratio** (prey 150/165/180 vs. chaser 175): BDH is robust (322/306/294). Second-order only competes at slow prey (336 at 150) and collapses at 165+ (140) as its parabola overshoots.
- **Memory width (Fourier features M)** (0/8/24/48): the *linear* associator ($M = 0$) is

optimal (306); the nonlinear expansion only adds variance (253–304). The relevant dynamics here are smooth, and a linear content-addressable memory captures them.

- **Learning rate** η (0.1/0.3/0.5/1.0): slow learning is best ($\eta = 0.1 \rightarrow 319$); *every* η beats first-order (280).
- **Weight decay** λ ($0/10^{-4}/10^{-3}/10^{-2}$): moderate decay is best ($10^{-3} \rightarrow 306$); every λ beats first-order.
- **Action discretization** (4/8/16 headings, model-free): linear-Q (~ 22), DQN (~ 37 –46), and PPO (~ 23 –30) stay far below the reflex (~ 179) at every discretization, so the model-free failure is *not* a discretization artifact — it persists at 16 actions.

6 Discussion

6.1 Why model-free value learning fails here

Three compounding factors explain the model-free collapse. First, the **deadly triad** (linear-Q is linear function approximation + TD bootstrapping + ϵ -greedy off-policy updates [4]) makes the value estimate unstable or divergent. Second, the **turn-rate clamp** means the greedy action cannot be executed instantaneously: even a perfect value function’s argmax heading takes ~ 0.1 s to reach, so one-step-greedy value learning systematically under-turns. Third, the **interception reward is inherently temporal**: catching a moving target requires anticipating a future position, which is a prediction problem, not a credit-assignment problem. A value function must propagate reward through many time steps of a high-frequency control loop to *implicitly* represent the same thing a world model *explicitly* predicts in one step.

The dense reward and strong function approximators (DQN, PPO) do not rescue this: they reliably learn to *approach* the prey but not to *intercept* it, and frequently do worse than the reflex.

6.2 Why the world model works

Interception is a prediction problem. The world model’s inductive bias — learn the map $s_t \mapsto v_{t+1}$ and roll it forward — is exactly aligned with the task, and its training signal (dense, self-supervised prediction error) is abundant at every tick, independent of the sparse catch reward. This is the Dreamer split [7] in miniature: the policy (steer at the forecast) is driven by outcome, while the model is driven by prediction. The fact that a *linear* associative memory suffices here suggests that, for smooth physical dynamics, content-addressable recall of local transitions is already a strong world model — and that the “fast weights” of the BDH architecture are best read as a predictive substrate.

6.3 Limitations

This is a controlled toy domain and a first step, not a claim of generality. Specifically:

1. The result is strongest where it should be — curved, predictable motion — and does not hold on genuinely unpredictable (Martingale) prey, where no predictor beats first-order.
2. On **reactive** prey (**flee**), the world model is only comparable to first-order; our imagination holds the chaser fixed while rolling the prey forward, which is a crude approximation of the closed-loop pursuit–evasion dynamics.

3. The world model is **linear**; the nonlinear (Fourier) expansion did not help on these smooth dynamics, so we do not demonstrate nonlinear world modeling.
4. The environment is single-prey, toroidal, and two-dimensional; real pursuit adds walls, multiple prey, partial observation, and sensing noise.
5. Baselines SAC (continuous control) and a circle-fit analytic predictor remain for future work, as do comparisons on a standard RL benchmark rather than a purpose-built task.

6.4 What would make this a full paper

To elevate this from a technical report to a full paper we would: (i) fix the closed-loop imagination so reactive prey are handled; (ii) add SAC and a circle-fit (or Kalman-with-turn) analytic baseline to close the baseline set; (iii) sweep stochasticity and noise to establish the boundary of “predictable enough”; and (iv) reproduce the core value-vs-world-model contrast on a standard interception benchmark with multiple seeds and significance tests. The conceptual claim — fast weights predict, not evaluate — does not depend on these, but its generality does.

7 Conclusion

We asked whether the fast-weight associative memory at the heart of the Dragon Hatchling architecture is a value function or a world model. In a seeded interception benchmark, the answer is decisive: the *same* memory, used to predict, reaches $\sim 94\%$ of the lead-pursuit ceiling and beats first- and second-order analytic extrapolation on curved prey, while used to evaluate it collapses below a no-prediction reflex — and even strong model-free learners (DQN, PPO) cannot match the reflex. The result supports a clean reading of fast-weight memories as predictive substrates trained by dense self-supervised error, with sparse outcome-driven planning on top — a Dreamer-style division of labor instantiated by a biologically plausible three-factor Hebbian rule. We hope this reframing is useful both to readers of the BDH literature and to practitioners choosing what to put in a fast-weight memory: put the *next state*, not the *value*.

Acknowledgments and disclosures

Author responsibilities. All authors take full responsibility for the content of this manuscript, regardless of how it was produced.

Use of generative AI language tools. Portions of this manuscript were drafted and edited with the assistance of a text-to-text generative AI language model. All results, code, references, and claims were reviewed and verified by the authors, who accept full responsibility for them; the AI tool is not an author. This statement is included to comply with arXiv’s policy on authors’ use of generative AI language tools (<https://info.arxiv.org/help/moderation/index.html>).

Code and data availability. The benchmark, all experiment configurations, and the code to reproduce every number in this paper are available at <https://github.com/llvm-x86/fast-weights-predict>.

References

- [1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, and M. Bartoszkiewicz. The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507, 2025.
- [2] J. Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- [3] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [4] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [5] R. S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.
- [6] N. Frémaux and W. Gerstner. Neuromodulated spike-timing-dependent plasticity, and theory of three-factor learning rules. *Frontiers in Neural Circuits*, 9:85, 2016.
- [7] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. arXiv:2301.04104, 2023.
- [8] B. Widrow and M. E. Hoff. Adaptive switching circuits. In *IRE WESCON Convention Record*, 1960.
- [9] N. A. Shneydor. *Missile Guidance and Pursuit: Kinematics, Dynamics and Control*. Horwood Publishing, 1998.
- [10] P. J. Nahin. *Chases and Escapes: The Mathematics of Pursuit and Evasion*. Princeton University Press, 2007.
- [11] B. R. Fajen and W. H. Warren. Behavioral dynamics of steering, obstacle avoidance, and route selection. *Journal of Experimental Psychology: Human Perception and Performance*, 29(2):343–362, 2003.